

在底层代码中，缓存到 KV Cache 中的 \$K_1\$ 和 \$V_1\$ 绝对不是汉字或字符串，而是由大量浮点数构成的高维矩阵(张量 **Tensor**)。

你可以把它们理解为模型对“我”这个字经过深度神经网络提炼后，形成的一组数学特征向量。为了让你直观理解，我们用一个**极度简化版的“玩具模型”**来举例说明它们究竟长什么样。

1. 它们的三维结构 (Shape)

在真实的 LLM(如 LLaMA-2)中，\$K_1\$ 和 \$V_1\$ 是存储在显存中的矩阵，通常包含以下几个维度：

- **Num_Heads**(注意力头数): 模型会从多个不同角度去理解这个字(比如有的头关注语法，有的头关注情感)。
- **Seq_Len**(序列长度): 当前是第几个字(对于单字“我”，长度就是 1)。
- **Head_Dim**(头部维度大小): 每个角度用多少个数字来描述。

假设我们的玩具模型有 2 个注意力头，每个头用 4 个特征维度，那么对于“我”这个字，它缓存的 \$K_1\$ 和 \$V_1\$ 大致长这样：

2. 举例: \$K_1\$ 的长相 (Key: 我是谁？我的特征是什么？)

\$K_1\$ 相当于“我”这个字的名片或目录索引，用于告诉后面的字：“如果你需要找主语、或者找第一人称，请来看我”。

Python

```
K1_for_我 = [  
    # 第 1 个注意力头(可能负责提取“语法”特征:发现这是一个代词/主语)  
    [[ 0.85, -0.12, 0.04, 0.99 ]],  
  
    # 第 2 个注意力头(可能负责提取“语义”特征:发现它代表“自身/自己”)  
    [[-0.55, 0.77, -0.21, 0.11 ]]  
]
```

3. 举例: \$V_1\$ 的长相 (Value: 我的具体内容是什么？)

\$V_1\$ 相当于“我”这个字的实体内容。当后面的字(比如“爱”)通过 \$Q_2\$ 匹配到了 \$K_1\$ 后，模型就会把 \$V_1\$ 里的数值按比例提取出来，融合到后面的计算中。

Python

```
V1_for_我 = [  
    # 第 1 个注意力头的具体内容值  
    [[ 0.10, 0.20, -0.44, 0.66 ]],  
]
```

```
# 第 2 个注意力头的具体内容值
[[ 0.33, -0.88, 0.15, 0.05]]
]
```

(注意:在物理显存中,它们就是这样密密麻麻的 16位浮点数 (FP16/BF16) 数组。)

4. 为什么要存这些数字?(结合你课程里的内存瓶颈)

当模型接着生成“爱”时(Step 2), 它会计算出“爱”的 Q_2 (查询向量)。

1. Q_2 会去和刚才存好的 K_1 做点积乘法(计算匹配度)。
2. 算完发现匹配度很高, 就会把 V_1 的特征提取过来, 和“爱”自己的特征融合。

致命的显存开销就在这里:

在真实的 7B 大模型中, 上面玩具模型里的 [2个头, 4个维度] 实际上是 [32个头, 128个维度]。这意味着, 每生成一个 **token**, 它的每一层神经网络都要缓存 $32 \times 128 = 4096$ 个浮点数。如果模型有 32 层, 存一个字的 KV Cache 就需要大约 256 KB。

如果在并发场景下(几百个人同时在让模型生成几千字的文章), 这些无穷无尽的浮点数矩阵就会把 A100 的 80GB 显存彻底塞满——这就是你课程资料中第一章强调 “**KV Cache** 是显存杀手” 以及必须引入 **PagedAttention**(按需分页存储这些数字块) 的根本原因。